

A Project Report
On
Smart traffic sign detection system

*Submitted in partial fulfillment of the
requirement for the award of the degree of*

MASTER OF COMPUTER APPLICATION



(Established under Galgotias University Uttar Pradesh Act No. 14 of 2011)

DEGREE

Session 2024-25
in

Computer Science and Engineering

By
Prashant pandey 24132030314
Pritam kumar 24132030326
Abhay kumar 24132030006

Under the guidance of
Dr.suneet gupta

SCHOOL OF COMPUTER APPLICATIONS AND TECHNOLOGY
GALGOTIAS UNIVERSITY, GREATER NOIDA
INDIA
Jan, 2025



SCHOOL OF COMPUTER APPLICATIONS AND
TECHNOLOGY
GALGOTIAS UNIVERSITY, GREATER NOIDA

CANDIDATE'S DECLARATION

I/We hereby certify that the work which is being presented in the project, entitled “**Smart traffic sign detection system**” in partial fulfillment of the requirements for the award of the MCA (Master of Computer Application) submitted in the School of Computer Applications and Technology of Galgotias University, Greater Noida, is an original work carried out during the period of August, 2023 to Jan and 2024, under the supervision of Dr Suneet gupta, Department of Computer Science and Engineering/School of Computer Applications and Technology , Galgotias University, Greater Noida.

The matter presented in the thesis/project/dissertation has not been submitted by me/us for the award of any other degree of this or any other places.

Student Names (Admission No.)

Prashant pandey	24SCSE2030176
Pritam kumar	24SCSE2030177
Abhay kumar	24SCSE2030079

This is to certify that the above statement made by the candidates is correct to the best of my knowledge.

Dr.Suneet gupta

Designation

CERTIFICATE

This is to certify that Project Report entitled
“.....” which is
submitted by in partial fulfillment of the requirement for the
award of degree MCA. in Department of of School of
Computer Applications and Technology, Galgotias University, Greater Noida, India is a
record of the candidate own work carried out by him/them under my supervision. The
matter embodied in this thesis is original and has not been submitted for the award of any
other degree

Signature of Examiner(s)

Signature of Supervisor(s)

Date: Jan, 2025

Place: Greater Noida

Abstract: Traffic Sign Detection (TSD) is a critical component of modern driver assistance systems and autonomous vehicles. This technology aims to automatically detect and classify traffic signs from road images, providing real-time information to drivers and onboard systems to enhance road safety and navigation efficiency.

Key Objectives:

1. Detection: Identifying the presence of traffic signs in a variety of environmental conditions, including different lighting, weather, and occlusions.

2. Classification: Categorizing detected signs into predefined classes such as stop signs, speed limits, and warning signs.

Methodology:

TSR systems typically use a combination of computer vision techniques and machine learning algorithms. The process involves several stages:

- **Image Preprocessing:** Enhancing image quality and reducing noise to improve detection accuracy.
- **Feature Extraction:** Using methods such as Histogram of Oriented Gradients (HOG) or Convolutional Neural Networks (CNNs) to extract relevant features from the images.
- **Detection:** Implementing algorithms like the Hough Transform or CNN-based object detectors to identify traffic sign regions.
- **Classification:** Applying machine learning models, particularly deep learning networks, to classify the detected signs accurately.
- **Speech Integration for Driver Assistance:** Text-to-Speech (TTS) technologies, such as Google's TTS API [3], have been effectively integrated into similar systems to provide real-time auditory feedback.

Applications: TSR enhances Advanced Driver Assistance Systems (ADAS) by providing real-time sign recognition, which assists drivers in obeying traffic laws and increasing overall road safety.

Additionally, TSR is crucial for the development of fully autonomous vehicles, enabling them to understand and respond to traffic signs without human intervention.

Conclusion:

Traffic Sign Recognition is a pivotal technology in the advancement of intelligent transportation systems, contributing significantly to safer and more efficient driving experiences. Continued research and innovation in this field hold the promise of achieving fully autonomous navigation and enhanced road safety.

Table of Contents

1. **Chapter 1: Introduction**
 - 1.1 Problem Introduction
 - 1.1.1 Motivation
 - 1.1.2 Project Objective
 - 1.1.3 Scope of the Project
 - 1.2 Related Previous Work
2. **Chapter 2: Literature Survey**
 - 2.1 Traditional Image Processing Techniques
 - 2.2 Machine Learning-Based Approaches
 - 2.3 Deep Learning and Convolutional Neural Networks (CNNs)
 - 2.4 YOLO and Real-Time Object Detection
 - 2.5 Speech Integration for Driver Assistance
 - 2.6 Challenges in Adverse Weather Conditions
3. **Chapter 3: Requirements**
 - 3.1 System Requirements
 - 3.2 Dependencies
 - 3.3 Proposed Algorithm
 - 3.4 Why did CNN use picture processing?
 - 3.5 Where is the CNN algorithm used?
 - 3.6 What is the CNN algorithm?
 - 3.7 CNN Algorithm Flow Chart
4. **Chapter 4: System Design and Methodology**
 - 4.1 Dataset Collection & Annotation
 - 4.1.1 Image Collection
 - 4.1.2 Annotation
 - 4.1.3 Model Training
 - 4.1.4 Setting Dataset YAML File
 - 4.1.5 Installing Required Libraries
 - 4.1.6 Training YOLOv8 Model
 - 4.2 Real-time Implementation
 - 4.2.1 Real-time Detection with OpenCV
 - 4.2.2 Block Diagram
 - 4.2.3 Architecture Diagram
 - 4.3 Speech Output for Detected Signs
 - 4.3.1 Generating Speech Files
 - 4.3.2 Playing Audio for Detected Signs
 - 4.4 System Architecture
 - 4.5 Data Flow Diagram (DFD)
 - 4.6 Class Diagram
 - 4.7 Flowchart
 - 4.8 Entity-Relationship (ER) Diagram
 - 4.9 Algorithms
5. **Chapter 5: Conclusion**
6. **Chapter 6: Project file structure and codes**

CHAPTER 1

INTRODUCTION

In the modern era, road safety has become a crucial aspect of transportation systems. One of the leading causes of road accidents is the failure to recognize traffic signs, particularly under challenging conditions like fog or poor visibility. To address this issue, this project focuses on developing a Smart Traffic Sign Detection System that utilizes cutting-edge computer vision techniques to enhance driver awareness and safety.

The core of this project lies in leveraging the YOLOv8 model, a highly efficient object detection framework developed by Ultralytics. The model has been meticulously trained using a diverse dataset comprising various classes of traffic signs and images captured under different environmental conditions. The system not only detects traffic signs in real-time through a camera but also displays their names on the screen and audibly announces them, ensuring accessibility and usability. Furthermore, it is optimized to function effectively even in foggy conditions, enhancing its reliability in adverse weather.

1.1 Problem Introduction

1.1.1 Motivation

The increasing rate of road accidents due to missed or misinterpreted traffic signs serves as a strong motivation for this project. The need for an intelligent system that assists drivers by identifying and communicating traffic sign information is paramount. Additionally, the challenge of ensuring detection accuracy in adverse conditions like fog further inspires the development of this system.

1.1.2 Project Objective

The primary objective of this project is to design and implement a system capable of:

1. Detecting and recognizing traffic signs in real time using a YOLOv8-based model.
2. Displaying the identified traffic sign names on the screen and audibly announcing them.
3. Maintaining high detection accuracy under challenging conditions such as fog.

1.1.3 Scope of the Project

This system is intended to be a crucial aid for drivers, contributing to road safety by reducing human error and enhancing situational awareness. Its applications can extend to autonomous

vehicles, driving assistance systems, and traffic management. The robust nature of the system ensures adaptability across various environments and conditions.

1.2 Related Previous Work

Traffic sign recognition has been an area of interest for researchers over the years. Early systems relied on traditional image processing techniques, which often lacked robustness in dynamic and challenging conditions. The introduction of deep learning and object detection models, such as YOLO, revolutionized this field by providing higher accuracy and efficiency. Previous work in this domain has demonstrated the efficacy of convolutional neural networks (CNNs) in detecting traffic signs; however, the ability to operate reliably in low-visibility scenarios remains a limitation in many systems. This project aims to bridge that gap by combining the latest YOLOv8 advancements with custom training on diverse datasets.

CHAPTER 2

Literature Survey

Traffic sign detection and recognition have been extensively studied due to their critical role in enhancing road safety and enabling advanced driver-assistance systems (ADAS). Over the years, researchers have developed numerous techniques and algorithms to address the challenges in this field, such as varying lighting conditions, partial occlusions, and diverse sign types. This literature survey explores existing methodologies and their contributions, focusing on advancements in object detection frameworks, especially those relevant to real-time systems.

2.1. Traditional Image Processing Techniques

Traditional image processing methods relied on feature extraction techniques like edge detection, color segmentation, and shape analysis to identify traffic signs. Methods such as Hough Transform were commonly used for detecting circular and triangular signs. While these techniques were computationally efficient, they lacked robustness against variations in lighting and environmental conditions.

2.2. Machine Learning-Based Approaches

With the rise of machine learning, researchers began employing classifiers like Support Vector Machines (SVM) and Random Forests combined with hand-crafted features such as Histogram of Oriented Gradients (HOG) and Scale-Invariant Feature Transform (SIFT). These methods improved detection accuracy but were limited in scalability and generalization to diverse datasets.

2.3. Deep Learning and Convolutional Neural Networks (CNNs)

The advent of deep learning, particularly CNNs, revolutionized traffic sign recognition. Models like AlexNet and VGGNet demonstrated significant improvements in accuracy by automatically learning hierarchical features. However, these models often required large computational resources and were unsuitable for real-time applications.

2.4. YOLO and Real-Time Object Detection

The introduction of the "You Only Look Once" (YOLO) framework marked a significant advancement in real-time object detection. YOLOv1, proposed by Redmon et al. [1], used a single neural network to predict bounding boxes and class probabilities simultaneously. Subsequent iterations, such as YOLOv3 and YOLOv4, further enhanced detection speed and accuracy. YOLOv8, the latest version by Ultralytics [2], offers improved performance through advanced architectures and optimization techniques, making it highly suitable for real-time traffic sign detection applications.

2.5. Speech Integration for Driver Assistance

Incorporating speech functionality into traffic sign detection systems enhances their usability by providing auditory feedback. This approach is particularly beneficial for drivers who may be unable to focus on visual cues due to challenging road conditions. Text-to-Speech (TTS) technologies, such as Google's TTS API [3], have been effectively integrated into similar systems to provide real-time auditory feedback.

2.6. Challenges in Adverse Weather Conditions

Foggy conditions present unique challenges for traffic sign detection systems due to reduced visibility and contrast. Several studies have addressed this issue by leveraging pre-processing techniques, such as histogram equalization and dehazing algorithms [4]. Additionally, the use of deep learning models trained on augmented datasets with fog-like conditions has shown promise in improving detection accuracy.

The literature highlights the evolution of traffic sign detection from traditional image processing methods to modern deep learning frameworks like YOLO. While traditional methods were computationally efficient, they lacked robustness and generalization. Machine learning approaches introduced feature-based classification, but their reliance on hand-crafted features limited scalability. Deep learning models like CNNs and YOLO have significantly enhanced detection accuracy and real-time performance. Speech integration and handling adverse conditions, such as fog, further emphasize the practical challenges and innovations in this domain. This project builds upon these advancements by utilizing the YOLOv8 model and incorporating auditory feedback for a robust and user-friendly traffic sign detection system.

Chapter 3

Requirements

3.1. System Requirements

- processor (CPU): Multi-core, e.g., MAC Air m1 or later, Intel i5/i7 or AMD Ryzen 5/7.
- GPU (Recommended): NVIDIA GPU with CUDA (e.g., GTX 1650, RTX 3050 or better).
- RAM: Minimum 8 GB, recommended 16 GB.
- Storage: SSD with at least 100 GB free space.
- Camera: 720p or 1080p webcam (e.g., Logitech C270 or C920).
- Operating System: Linux, Windows, or macOS.
- Additional Hardware: Speakers/microphone for TTS output.
- Internet Connection: Required for gTTS.
- Software Requirements: NVIDIA drivers, CUDA, and cuDNN (if using GPU).

3.2. Dependencies:

- Python 3.8 or higher
- Flask
- OpenCV (opencv-python-headless for servers)
- YOLOv8 (ultralytics library)
- gTTS (gTTS library for text-to-speech)

3.3. Proposed Algorithm:

Convolutional Brain Organization (Convolutional Brain Organization) is a state of profound concentrating on brain organization. So, consider CNN as a framework acquiring information on set of decides that can take a picture input, dole out importance (versatile loads and predispositions) to unique capabilities/objects in the image, and recognize them from each different. Rhonchus works by extracting functions from films.

Gray scale method is used inside the preceding picture to convert the files. After body conversion, every factor is compared with the teach dataset to stumble on suspicious hobby.

3.4. Why did CNN use picture processing?

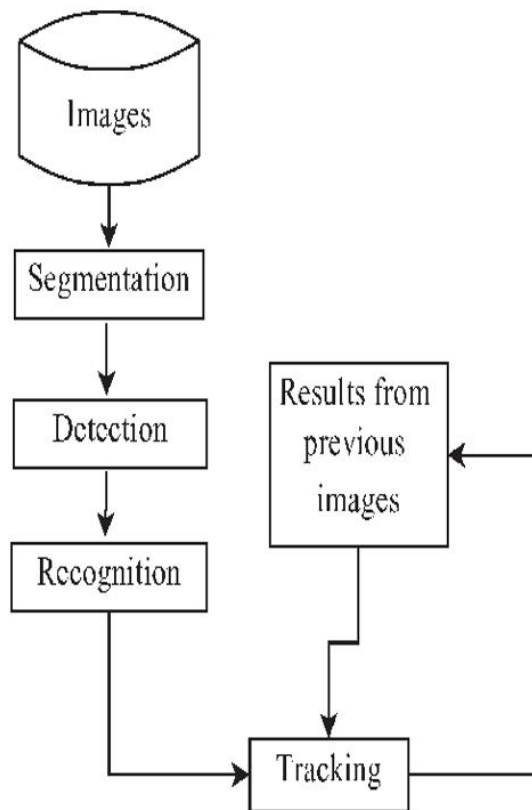
Convolutional Brain Organizations (CNN) are a subtype of brain organizations which are in general used in photograph and speech recognition applications. Its built-in convolutional layers reduce excessive-dimensional photos without losing facts. That is why they're suitable for this application.

3.5. Where is the CNN algorithm used?

Applications in image and video popularity, recommender systems, photo class, photo segmentation, clinical photo evaluation, herbal language processing, mind interfaces, and economic time collection.

3.7.CNN algorithm flow chart

BLOCK DIAGRAM



CHAPTER 4

SYSTEM DESIGN AND METHODOLOGY

This project focuses on developing a real-time Traffic Sign Detection System to enhance road safety. The key components include:

4.1.1.Dataset Collection & Annotation: Gather images of various traffic signs from public datasets and manually annotate them to create a labeled dataset

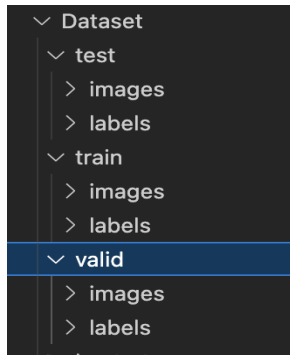
4.1.2Image collection: Images of traffic signs were sourced from **GTSRB (German Traffic Sign Recognition Benchmark)**.



4.1.3. Annotation

- **LabelImg** was used to annotate the dataset.
- The annotations were saved in **YOLO format** (each image has a `.txt` file containing bounding box coordinates).

That is how the training dataset and its structure looks like



project-folder/

```
├── train/
│   ├── images/
│   │   ├── img1.jpg
│   │   ├── img2.jpg
│   │   └── ...
│   └── labels/
│       ├── img1.txt
│       ├── img2.txt
│       └── ...
├── valid/
│   ├── images/
│   └── labels/
└── test/
    ├── images/
    └── labels/
```

4.1.4.Model Training: Utilize the YOLOv8 model, known for its efficiency in object detection, and train it on the annotated dataset to accurately identify and classify traffic signs.

4.1.5.Setting Dataset YAML File

```
Train: /Users/prashantpandey/downloads/datasets/train/images
val: /Users/prashantpandey/downloads/datasets/valid/images
test: /Users/prashantpandey/downloads/datasets/test/images

nc: 35
names: ['GREEN LIGHT', 'RED LIGHT', 'Speed Limit 10', 'Speed Limit 100', 'Speed Limit 110', 'Speed Limit 120', 'Speed Limit 20',
'Speed Limit 30', 'Speed Limit 40', 'Speed Limit 50', 'Speed Limit 60', 'Speed Limit 70', 'Speed Limit 80', 'Speed Limit 90', 'Stop no
entry', 'bus_stop', 'do_not_enter', 'do_not_stop', 'do_not_turn_l', 'do_not_turn_r', 'do_not_u_turn', 'enter_left_lane', 'green_light',
'left_right_lane', 'no_parking', 'parking', 'Pedestrian crossing', 'pedestrian_zebra_cross', 'railway_crossing', 'red_light', 'stop',
't_intersection_left', 'traffic_light', 'u_turn', 'warning', 'yellow_light']

roboflow:
  workspace: project-sign-detection
  project: traffic-sign-cdfml
  version: 2
  license: CC BY 4.0
  url: https://universe.roboflow.com/project-sign-detection/traffic-sign-cdfml/dataset/2
```

4.1.6.Installing Required Libraries

```
pip install ultralytics opencv-python gtts
```

4.1.7.Training YOLOv8 Model

In this project, we trained our traffic sign detection model using two primary approaches:

1. **Google Colab Platform:** Initially, we utilized Google Colab, a browser-based tool that allows users to write and execute Python code. Colab is particularly beneficial for machine learning and data analytics tasks, offering free access to computing resources, including GPUs. The platform provides various GPU types, such as NVIDIA Tesla K80, T4, and P100, with configurations that may change over time. This flexibility makes Colab a valuable resource for developing and testing machine learning models.
2. **Local GPU Training:** To further enhance performance and have more control over the training environment, we transitioned to training the model on a local terminal equipped with a dedicated GPU. This approach allowed us to utilize different datasets and fine-tune the model more effectively. By leveraging the local GPU, we achieved faster training times and improved model accuracy, as we could customize the hardware and software settings to better suit our specific requirements

➤ Run the following command to train YOLOv8:

yolo task=detect mode=train model=yolov8n.pt data=data.yaml epochs=100 imgsz=100
batch=50 device=0

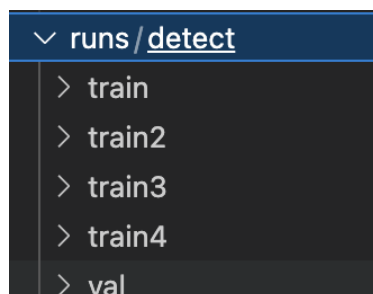
- That is how the model is trained inside the terminal:

```
Using 0 dataloader workers
Logging results to runs/detect/train4
Starting training for 50 epochs...
```

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
1/50	0G	1.017	4.077	1.247	37	640: 1
	Class	Images	Instances	Box(P	R	mAP50 m
	all	488	529	0.265	0.525	0.166
0.129						
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
2/50	0G	0.987	3.155	1.228	23	640: 1
	Class	Images	Instances	Box(P	R	mAP50 m
	all	488	529	0.324	0.512	0.394
0.304						
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
3/50	0G	0.9683	2.583	1.19	32	640: 1
	Class	Images	Instances	Box(P	R	mAP50 m
	all	488	529	0.411	0.646	0.553
0.437						
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
4/50	0G	0.9266	2.203	1.158	30	640: 1

- The model was trained for **50 epochs** with a batch size of **8**.
- The best weights were saved at `runs/detect/train/weights/best.pt`.

➤ That is how the result of the training looks like:



4.2 Real-time Implementation

4.2.1 Real-time Detection with OpenCV

```
import cv2
```

```

import time

def detect_traffic_sign():

    cap = cv2.VideoCapture(0)

    while True:

        ret, frame = cap.read()

        results = model(frame)

        cv2.imshow("Traffic Sign Detection", frame)

        if cv2.waitKey(1) & 0xFF == ord('q'):

            break

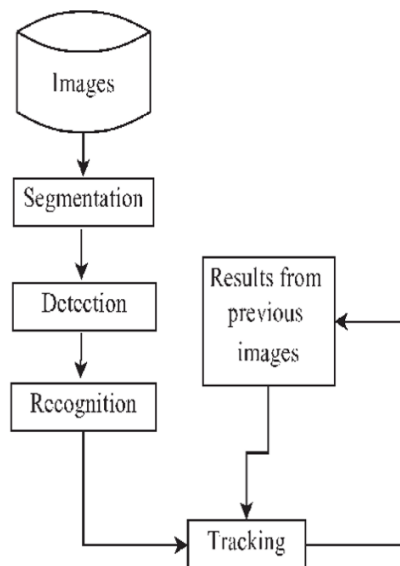
    cap.release()

    cv2.destroyAllWindows()

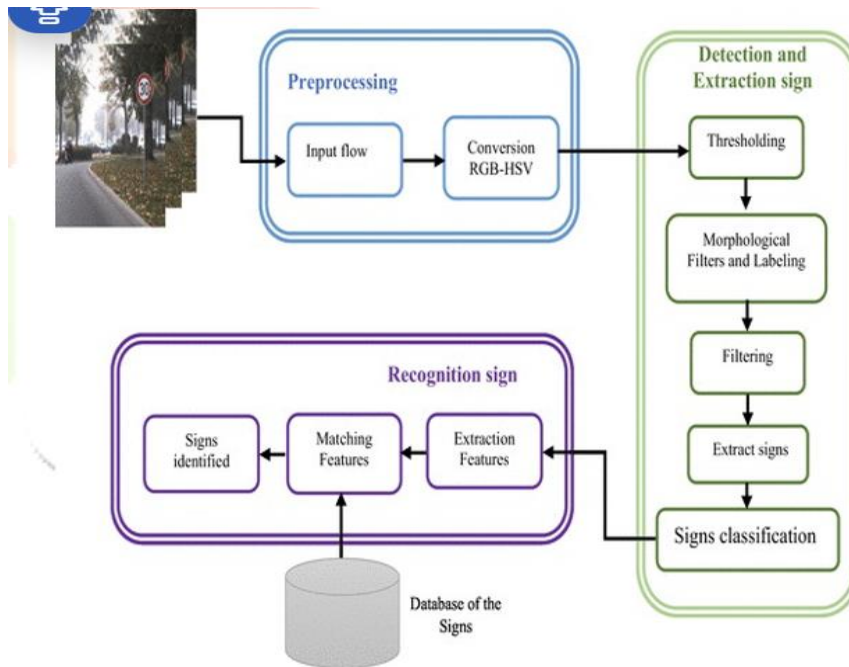
detect_traffic_sign()

```

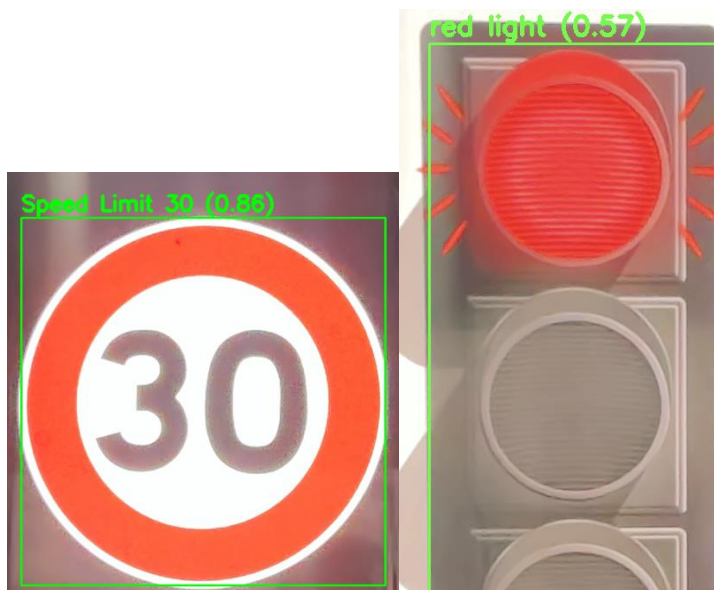
4.2.2.Block diagram:



4.2.3. Architecture diagram:



- That is how it shows the detection result on the screen:



4.3. Speech Output for Detected Signs

4.3.1 Generating Speech Files

```
from gtts import gTTS

import os

LABELS = ["GREEN LIGHT", "RED LIGHT", "Speed Limit 10", "Speed Limit 100"]

SPEECH_DIR = "speech_files"

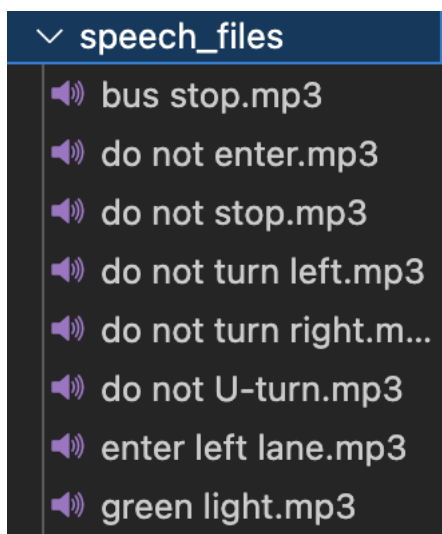
os.makedirs(SPEECH_DIR, exist_ok=True)

for label in LABELS:

    tts = gTTS(text=label, lang='en')

    tts.save(os.path.join(SPEECH_DIR, f'{label}.mp3'))
```

That is how the generated audio files look like:



4.3.2 Playing Audio for Detected Signs

```
import subprocess
```

```
def speak(text):

    filename = f"speech_files/{text}.mp3"

    subprocess.run(["mpg123", filename], stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
```

Designing a comprehensive system for traffic sign detection and recognition involves several components and methodologies. Below is an overview of the system design, including architecture, data flow diagrams (DFD), class diagrams, flowcharts, and algorithms pertinent to your project.

4.4. System Architecture:

The system architecture outlines the high-level structure of the traffic sign detection and recognition system. It includes the main components and their interactions:

- **Data Acquisition:** Capturing real-time video frames from a camera.
- **Pre-processing:** Enhancing image quality and preparing data for analysis.
- **Detection Module:** Utilizing YOLO models to identify potential traffic signs in frames.
- **Recognition Module:** Classifying detected signs into specific categories.
- **Alert System:** Providing audio feedback to the user about detected signs.

4.5.Data Flow Diagram (DFD):

A DFD illustrates the flow of information within the system:

- **Level 0 (Context Diagram):** Depicts the system as a single process interacting with external entities (e.g., User, Camera).
- **Level 1:** Breaks down the main process into sub-processes:
 - **1.1 Data Acquisition:** Capturing frames from the camera.
 - **1.2 Pre-processing:** Applying filters and adjustments to frames.
 - **1.3 Detection:** Running YOLO models to detect traffic signs.
 - **1.4 Recognition:** Classifying detected signs.
 - **1.5 Alert Generation:** Generating audio alerts for recognized signs.

4.6. Class Diagram:

The class diagram represents the static structure of the system, highlighting classes, attributes, methods, and relationships:

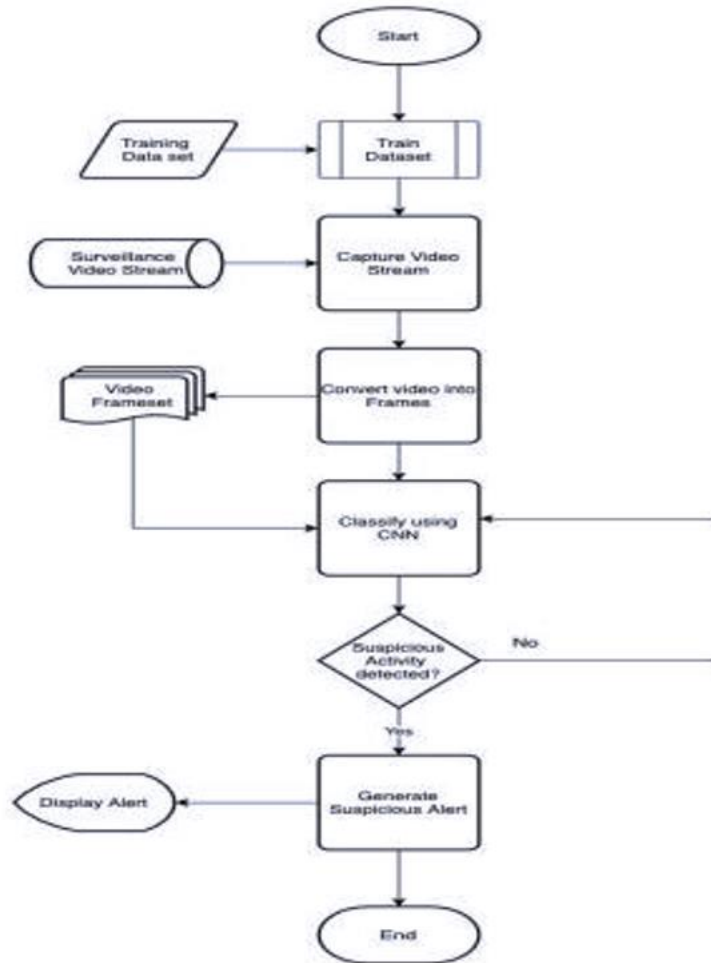
- **Classes:**
 - **Camera:** Attributes: resolution, frame_rate; Methods: capture_frame().
 - **ImageProcessor:** Attributes: filters; Methods: apply_preprocessing(frame).

- **YOLOModel:** Attributes: model_path; Methods: load_model(), detect_objects(frame).
- **TrafficSign:** Attributes: type, location; Methods: classify_sign().
- **AudioAlert:** Attributes: volume; Methods: generate_alert(sign_type).

4.7. Flowchart:

A flowchart details the step-by-step operational flow:

- **Start**
 - **Initialize Camera**
 - **Load YOLO Models**
 - **While True:**
 - **Capture Frame**
 - **Pre-process Frame**
 - **Detect Signs Using Model 1**
 - **Detect Signs Using Model 2**
 - **Combine and Filter Detections**
 - **For Each Detection:**
 - **Classify Sign**
 - **If New Detection:**
 - **Generate Audio Alert**
 - **Display Frame with Annotations**
 - **End Loop on User Input**
- **End**



4.8. Entity-Relationship (ER) Diagram:

The ER diagram showcases the relationships between entities:

- **Entities:**
 - **Camera**
 - **Frame**
 - **YOLOModel**
 - **TrafficSign**
 - **AudioAlert**
- **Relationships:**
 - **Camera** "captures" **Frame**
 - **YOLOModel** "processes" **Frame**
 - **Frame** "contains" **TrafficSign**
 - **TrafficSign** "triggers" **AudioAlert**

4.9. Algorithms:

- **Traffic Sign Detection Algorithm:**
 1. **Input:** Captured frame from the camera.
 2. **Pre-process:** Apply necessary image pre-processing techniques.
 3. **Detection:** Use YOLO models to detect potential traffic signs.
 4. **Post-process:** Apply Non-Maximum Suppression (NMS) to filter overlapping boxes.
 5. **Output:** List of detected traffic signs with bounding boxes and confidence scores.
- **Audio Alert Generation Algorithm:**
 1. **Input:** Detected traffic sign.
 2. **Check:** If the sign is new or has not been announced recently.
 3. **Generate:** Use text-to-speech to create an audio alert.
 4. **Output:** Play the audio alert to the user.

These components collectively provide a structured approach to designing and implementing a traffic sign detection and recognition system. Each diagram and algorithm offers a different perspective, ensuring a comprehensive understanding of the system's functionality and interactions.

CHAPTER 5.

CONCLUSION

Their paper shows the implementation of a convolutional neural network for traffic signal reputation. This has helped us research which components and situations govern the actual-world quality of the layout, in addition to the underlying community offerings and numerous enhancement capabilities. By adding more layers at the convolution level of the community to extract extra functions, we can reap better outcomes. It is feasible to try and sift via the anomalies within the heterogeneous and better training within the community muddle and extract the commonplace parts, so that the transformation of the picture transforms them

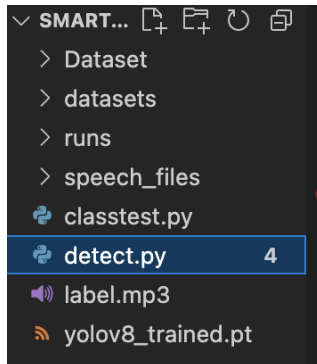
Future direction:-To further develop your traffic sign detection project, consider enhancing detection accuracy by implementing advanced models like Vision Transformers, which have shown promise in improving detection precision. Additionally, focus on optimizing real-time performance by adapting your models for deployment on embedded systems, crucial for applications like autonomous driving. Expanding and diversifying your dataset to include a broader range of traffic sign data from various regions and conditions will enhance model robustness. Integrating your detection system with other Advanced Driver Assistance Systems (ADAS), such as lane departure warnings and collision avoidance systems, can provide a comprehensive safety solution. Lastly, ensure compliance with local traffic laws and consider ethical implications, especially regarding data privacy and system reliability. By focusing on these areas, you can enhance the effectiveness and applicability of your traffic sign detection system in real-world scenarios.

CHAPTER 6

FILE STRUCTURE AND CODES

6.1. Project file structure

- That is how the file structure of the system looks like:



6.2. Codes

- All these codes used IN this project:

Detect.py

```
import cv2
import numpy as np
import os
import threading
import time
import subprocess
from ultralytics import YOLO
from gtts import gTTS

# Load both YOLO models
model1 = YOLO('yolov8_trained.pt') # First model
model2 = YOLO('runs/detect/train4/weights/best.pt') # Second model

# Define labels for both models
LABELS_1 = {
    0: 'GREEN LIGHT', 1: 'RED LIGHT', 2: 'Speed Limit 10', 3: 'Speed Limit 100',
    4: 'Speed Limit 110', 5: 'Speed Limit 120', 6: 'Speed Limit 20', 7: 'Speed Limit 30',
    8: 'Speed Limit 40', 9: 'Speed Limit 50', 10: 'Speed Limit 60', 11: 'Speed Limit 70',
    12: 'Speed Limit 80', 13: 'Speed Limit 90', 14: 'Stop no entry'
}

LABELS_2 = {
    0: 'bus stop', 1: 'do not enter', 2: 'do not stop', 3: 'do not turn right',
    4: 'do not turn left', 5: 'do not U-turn', 6: 'enter left lane', 7: 'green light',
    8: 'left-right lane', 9: 'no parking', 10: 'parking', 11: 'pedestrian crossing',
    12: 'pedestrian zebra crossing', 13: 'railway crossing', 14: 'red light',
    15: 'stop', 16: 'T-intersection left', 17: 'traffic light', 18: 'U-turn',
    19: 'warning', 20: 'yellow light'
}
```



```

# Directory for speech files
SPEECH_DIR = os.path.abspath("speech_files")
os.makedirs(SPEECH_DIR, exist_ok=True)

# Initialize speech control variables
last_spoken_label = None
last_spoken_time = 0
speech_delay = 2 # Seconds between speech outputs

# Function to pre-generate speech files
def generate_speech_files():
    for label in set(LABELS_1.values()).union(set(LABELS_2.values())):
        filename = os.path.join(SPEECH_DIR, f'{label}.mp3')
        if not os.path.exists(filename): # Avoid regenerating
            tts = gTTS(text=label, lang='en')
            tts.save(filename)
            print(f"□ Generated: {filename}")

generate_speech_files() # Pre-generate speech files

# Function to play the pre-generated speech file
def speak(text):
    filename = os.path.join(SPEECH_DIR, f'{text}.mp3')
    if os.path.exists(filename):
        print(f"□ Playing: {text}") # Debugging
        subprocess.run(["mpg123", filename], stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL) #
    Works better than os.system()
    else:
        print(f"✗Error: {filename} not found!")

# Apply Non-Maximum Suppression (NMS)
def apply_nms(detections, threshold=0.5):
    if len(detections) == 0:
        return []

    boxes = np.array([d[:4] for d in detections])
    scores = np.array([d[4] for d in detections])

    indices = cv2.dnn.NMSBoxes(boxes.tolist(), scores.tolist(), 0.5, threshold)

    if indices is None or len(indices) == 0:
        return []
    if isinstance(indices, tuple):
        indices = indices[0] # Extract list if tuple

    return [detections[i] for i in indices.flatten()]

# Process a frame using both models
def process_frame(frame):
    global last_spoken_label, last_spoken_time

    try:
        # Run both models on the frame
        results1 = model1.predict(frame)
        results2 = model2.predict(frame)

```

```

detections = []

# Process results from first model
for result in results1:
    for box in result.bboxes:
        label_index = int(box.cls)
        conf = float(box.conf)
        if conf > 0.5:
            label_text = LABELS_1.get(label_index, "Unknown")
            x1, y1, x2, y2 = map(int, box.xyxy[0])
            detections.append((x1, y1, x2, y2, conf, label_text))

# Process results from second model
for result in results2:
    for box in result.bboxes:
        label_index = int(box.cls)
        conf = float(box.conf)
        if conf > 0.5:
            label_text = LABELS_2.get(label_index, "Unknown")
            x1, y1, x2, y2 = map(int, box.xyxy[0])
            detections.append((x1, y1, x2, y2, conf, label_text))

# Apply NMS to remove duplicate bounding boxes
filtered_detections = apply_nms(detections)

# Speak out the first detected sign
if filtered_detections:
    _, _, _, _, current_text = filtered_detections[0]
    current_time = time.time()

    if current_text != last_spoken_label and (current_time - last_spoken_time > speech_delay):
        print(f"□ Speaking: {current_text}")
        tts_thread = threading.Thread(target=speak, args=(current_text,))
        tts_thread.start()
        last_spoken_label = current_text
        last_spoken_time = current_time

# Draw bounding boxes on the frame
for x1, y1, x2, y2, conf, label_text in filtered_detections:
    cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
    cv2.putText(frame, f"{label_text} ({conf:.2f})", (x1, y1 - 10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

except Exception as e:
    print(f"✗Error processing frame: {e}")

return frame

# Main function to start video capture
def main():
    cap = cv2.VideoCapture(0) # Open camera

    if not cap.isOpened():
        print("✗Error: Cannot access the camera.")
        return

```

```

while True:
    ret, frame = cap.read()
    if not ret:
        print("✗Error: Failed to capture frame from the camera.")
        break

    # Process the frame with both models
    frame = process_frame(frame)

    # Display the frame
    cv2.imshow("Traffic Sign Detection", frame)

    # Exit on pressing 'q'
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()

# Run the main function
if __name__ == "__main__":
    main()

```

➤ ***To know the classes of the dataset:***

```

from ultralytics import YOLO

model = YOLO('runs/detect/train4/weights/best.pt' )

print(model.names)

```